

4 DeepProbLog Inference

This section explains how a DeepProbLog model is used for a given query at prediction time.

ProbLog Inference As inference in DeepProbLog closely follows that in ProbLog, we now summarize ProbLog inference using the burglary example explained before. For full details, we refer to Fierens et al. [2015]. The program describing the example is explained in Section 3, Equation (1). We can query the program for the probabilities of given query atoms, say, the single query `calls(mary)`. ProbLog inference proceeds in four steps. (i) The first step grounds the logic program with respect to the query, that is, it generates all ground instances of clauses in the program the query depends on. The grounded program for query `calls(mary)` is shown in Figure 1a. (ii) The second step rewrites the ground logic program into a formula in propositional logic that defines the truth value of the query in terms of the truth values of probabilistic facts. In our example, the resulting formula is `calls(mary)  $\leftrightarrow$  hears_alarm(mary)  $\wedge$  (burglary  $\vee$  earthquake)`. (iii) The third step compiles the logic formula into a Sentential Decision Diagram (SDD, Darwiche [2011]), a form that allows for efficient evaluation of the query, using knowledge compilation technology [Darwiche and Marquis, 2002]. The SDD for our example is shown in Figure 1b, where rounded grey rectangles depict variables corresponding to probabilistic facts, and the rounded red rectangle denotes the query atom defined by the formula. The white rectangles correspond to logical operators applied to their children. (iv) The fourth and final step evaluates the SDD bottom-up to calculate the success probability of the given query, starting with the probability labels of the leaves as given by the program and performing addition in every or-node and multiplication in every and-node. The intermediate results are shown next to the nodes in Figure 1b, ignoring the blue numbers for now.

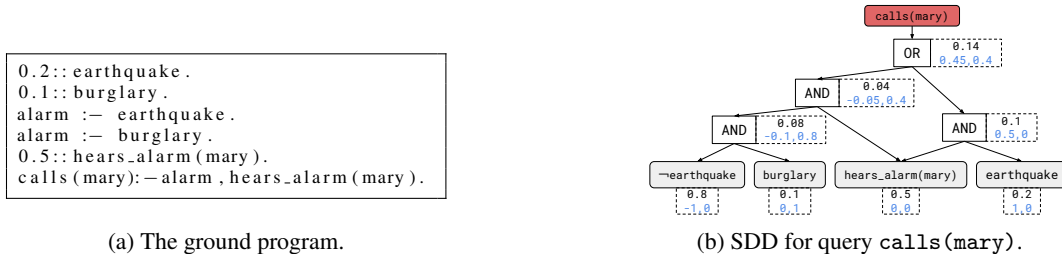


Figure 1: Inference in ProbLog.

DeepProbLog Inference Inference in DeepProbLog works exactly as described above, except that a forward pass on the neural network components is performed every time we encounter a neural predicate during grounding. When this occurs, the required inputs (e.g., images) are fed into the neural network, after which the resulting scores of their softmax output layer are used as the probabilities of the ground AD.

5 Learning in DeepProbLog

We now introduce our approach to jointly train the parameters of probabilistic facts and neural networks in DeepProbLog programs. We use the *learning from entailment* setting [De Raedt et al., 2016], that is, given a DeepProbLog program with parameters $\mathcal{X}$, a set $\mathcal{Q}$ of pairs (q, p) with q a query and p its desired success probability, and a loss function L , compute:

$$\arg \min_{\vec{x}} \frac{1}{|\mathcal{Q}|} \sum_{(q,p) \in \mathcal{Q}} L(P_{\mathcal{X}=\vec{x}}(q), p)$$

In contrast to the earlier approach for ProbLog parameter learning in this setting by Gutmann et al. [2008], we use gradient descent rather than EM, as this allows for seamless integration with neural network training. An overview of the approach is shown in Figure 2a. Given a DeepProbLog program, its neural network models, and a query used as training example, we first ground the program with respect to the query, getting the current parameters of nADs from the external models, then use the ProbLog machinery to compute the loss and its gradient, and finally use these to update the parameters in the neural networks and the probabilistic program.

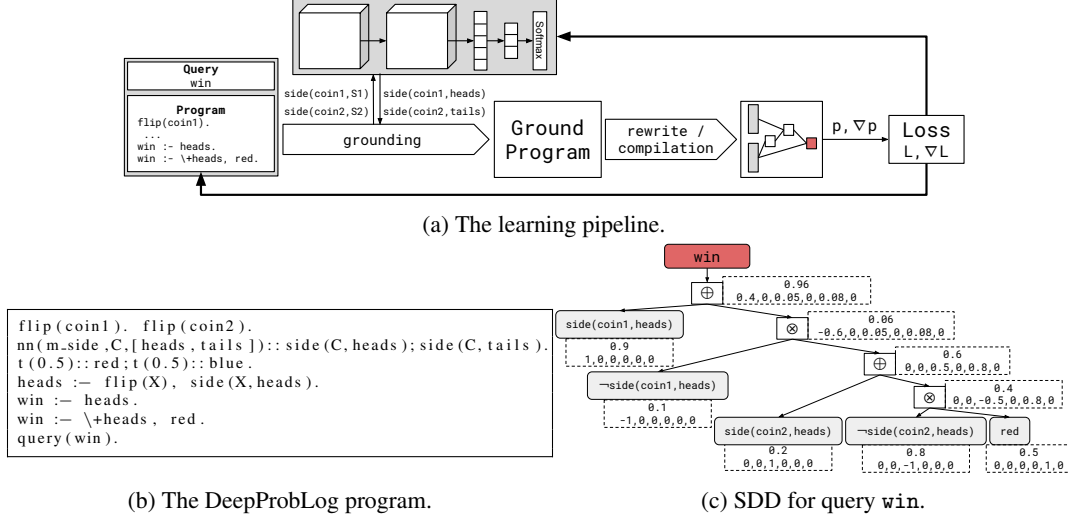


Figure 2: Parameter learning in DeepProbLog.

More specifically, to compute the gradient with respect to the probabilistic logic program part, we rely on Algebraic ProbLog (aProbLog, [Kimmig et al., 2011]), a generalization of the ProbLog language and inference to arbitrary commutative semirings, including the gradient semiring [Eisner, 2002]. In the following, we provide the necessary background on aProbLog, discuss how to use it to compute gradients with respect to ProbLog parameters and extend the approach to DeepProbLog.

aProbLog and the gradient semiring ProbLog annotates each probabilistic fact f with the probability that f is true, which implicitly also defines the probability that f is false, and thus its negation $\neg f$ is true. It then uses the probability semiring with regular addition and multiplication as operators to compute the probability of a query on the SDD constructed for this query, cf. Figure 1b. This idea is generalized in aProbLog to compute such values based on arbitrary commutative semirings. Instead of probability labels on facts, aProbLog uses a labeling function that explicitly associates values from the chosen semiring with both facts and their negations, and combines these using semiring addition $\oplus$ and multiplication $\otimes$ on the SDD. We use the gradient semiring, whose elements are tuples $(p, \frac{\partial p}{\partial x})$, where p is a probability (as in ProbLog), and $\frac{\partial p}{\partial x}$ is the partial derivative of that probability with respect to a parameter x , that is, the probability p_i of a probabilistic fact with learnable probability, written as $t(p_i) :: f_i$. This is easily extended to a vector of parameters $\vec{x} = [x_1, \dots, x_N]^T$, the concatenation of all N parameters in the ground program. Semiring addition $\oplus$, multiplication $\otimes$ and the neutral elements with respect to these operations are defined as follows:

$$(a_1, \vec{a}_2) \oplus (b_1, \vec{b}_2) = (a_1 + b_1, \vec{a}_2 + \vec{b}_2) \quad (2) \quad e^\oplus = (0, \vec{0}) \quad (4)$$

$$(a_1, \vec{a}_2) \otimes (b_1, \vec{b}_2) = (a_1 b_1, b_1 \vec{a}_2 + a_1 \vec{b}_2) \quad (3) \quad e^\otimes = (1, \vec{0}) \quad (5)$$

Note that the first element of the tuple mimics ProbLog’s probability computation, whereas the second simply computes gradients of these probabilities using derivative rules.

Gradient descent for ProbLog To use the gradient semiring for gradient descent parameter learning in ProbLog, we first transform the ProbLog program into an aProbLog program by extending the label of each probabilistic fact $p :: f$ to include the probability p as well as the gradient vector of p with respect to the probabilities of all probabilistic facts in the program, i.e.,

$$L(f) = (p, \vec{0}) \quad \text{for } p :: f \text{ with fixed } p \quad (6)$$

$$L(f_i) = (p_i, \mathbf{e}_i) \quad \text{for } t(p_i) :: f_i \text{ with learnable } p_i \quad (7)$$

$$L(\neg f) = (1 - p, -\nabla p) \quad \text{with } L(f) = (p, \nabla p) \quad (8)$$

where the vector $\mathbf{e}_i$ has a 1 in the i th position and 0 in all others. For fixed probabilities, the gradient does not depend on any parameters and thus is 0. For the other cases, we use the semiring labels as introduced above. For instance, assume we want to learn the probabilities of earthquake

and burglary in the example of Figure 1, while keeping those of the other facts fixed. Then, in Figure 1b, the nodes in the SDD now also contain the gradient (below, in blue). The result shows that the partial derivative of the proof query is 0.45 and 0.4 w.r.t. the earthquake and burglary parameters respectively. To ensure that ADs are always well defined, i.e., the probabilities of the facts in the same AD sum to one, we re-normalize these after every gradient descent update.

Gradient descent for DeepProbLog In contrast to probabilistic facts and ADs, whose parameters are updated based on the gradients computed by aProbLog, the probabilities of the neural predicates are a function of the neural network parameters. The neural predicates serve as an interface between the logic and the neural side, with both sides treating the other as a black box. The logic side can calculate the gradient of the loss w.r.t. the output of the neural network, but is unaware of the internal parameters. However, the gradient w.r.t. the output is sufficient to start backpropagation, which calculates the gradient for the internal parameters. Then, standard gradient-based optimizers (e.g. SGD, Adam, ...) are used to update the parameters of the network. During gradient computation with aProbLog, the probabilities of neural ADs are kept constant. Furthermore, updates on neural ADs come from the neural network part of the model, where the use of a softmax output layer ensures they always represent a normalized distribution, hence not requiring the additional normalization as for non-neural ADs. The labeling function for facts in nADs is

$$L(f_j) = (m_q(\vec{t})_j, \mathbf{e}_j) \quad \text{for } nn(m_q, \vec{t}, \vec{u}) :: f_i; \dots; f_k \text{ a nAD} \quad (9)$$

Example We will demonstrate the learning pipeline (shown in Figure 2a) using the following game. We have two coins and an urn containing red and blue balls. We flip both coins and take a ball out of the urn. We win if the ball is red, or at least one coin comes up heads. However, we need to learn to recognize heads and tails using a neural network, while only observing examples of wins or losses instead of explicitly learning from coin examples labeled with the correct side. We also learn the distribution of the red and blue balls in the urn. We show the program in Figure 2b. There are 6 parameters in this program: the first four originate from the neural predicates (heads and tails for the first and second coin). The last two are the logic parameters that model the chance of picking out a red or blue ball. During grounding, the neural network classifies coin1 and coin2. According to the neural network, the first coin is most likely heads ($p = 0.9$), and the second one most likely tails ($p = 0.2$). Figure 2c shows the corresponding SDD with the AND/OR nodes replaced with the respective semiring operations. The top number is the probability, and the numbers below are the gradient. On the top node, we can see that we have a 0.96 probability of winning, but also that the gradient of this probability is 0.4 and 0.05 w.r.t the probability of being heads for the first and second coin respectively, and 0.08 for the chance of the ball being red.

6 Experimental Evaluation

We perform three sets of experiments to demonstrate that DeepProbLog supports (i) symbolic and subsymbolic reasoning and learning, that is, both logical reasoning and deep learning; (ii) program induction; and (iii) both probabilistic logic programming and deep learning.

We provide implementation details at the end of this section and list all programs in Appendix A.

Logical reasoning and deep learning To show that DeepProbLog supports both logical reasoning and deep learning, we extend the classic learning task on the MNIST dataset (Lecun et al. [1998]) to two more complex problems that require reasoning:

- T1:** addition(, , 8): Instead of using labeled single digits, we train on pairs of images, labeled with the sum of the individual labels. The DeepProbLog program consists of the clause `addition(X,Y,Z) :- digit(X,X2), digit(Y,Y2), Z is X2+Y2.` and a neural AD for the `digit/2` predicate (this is shorthand notation for *of arity 2*), which classifies an MNIST image. We compare to a CNN baseline classifying the concatenation of the two images into the 19 possible sums.
- T2:** addition(, , , , 63): the input consists of two lists of images, each element being a digit. This task demonstrates that DeepProbLog generalizes well beyond training data. Learning the new predicate requires only a small change in the logic program. We train the model on single digit numbers, and evaluate on three digit numbers.